I am going to give you a few different set of coding rules. Can you help me create a single set of rules that accounts for everything in all the sets? I also want you to tell me if there are any ambiguities or contradictions

**claude-opus-4-6-v1**

I'd be happy to help you consolidate multiple sets of coding rules into a single unified set. Please go ahead and share all the different sets of rules, and I will:

1. **Merge** them into one comprehensive, well-organized set
2. **Identify any contradictions** where rules from different sets conflict with each other
3. **Flag any ambiguities** where rules are unclear or could be interpreted in multiple ways
4. **Highlight overlaps** where multiple sets say the same thing (so we can deduplicate)

Go ahead and paste the sets of rules whenever you're ready!

The first set of rules are as follows:

1. Class names of models, blocks and packages should start with an upper-case letter and be a noun or a noun with a combination of adjectives and nouns. Use camel-case notation to combine multiple words, such as `HeatTransfer`. Don't repeat higher level package names, for example, rather than `Chillers.CarnotChiller`, use `Chillers.Carnot`.
2. Instance names should be a combination of the first three characters of a word, such as `preDro` for pressure drop model. Where applicable, a single character can be used if this is generally understood, such as `T` for temperature, `p` for pressure, `u` for control input and `y` for control output signal. As needed, these can be augmented, for example a controller that outputs a control signal for a valve and a damper may output `yVal` and `yDam`.
3. The following variables are frequently used for physical quantities:
   - `T` for temperature,
   - `p` for pressure,
   - `dp` for pressure difference,
   - `P` for power,
   - `E` for energy (or `Q` for thermal energy),
   - `X` for mass fraction,
   - `Q_flow` for heat flow rate
   - `m_flow` for mass flow rate and
   - `H_flow` for enthalpy flow rate.
4. The nomenclature used in the package Buildings.Utilities.Psychrometrics is as

follows,

- Uppercase `X` denotes mass fraction per total mass.
- Lowercase `x` denotes mass fraction per mass of dry air.
- The notation `z_xy` denotes that the function or block has output `z` and inputs `x` and `y`.
- The symbol `pW` denotes water vapor pressure, `TDewPoi` denotes dew point temperature, `TWetBul` denotes wet bulb temperature, and `TDryBul` (or simply `T`) denotes dry bulb temperature.

5. Control input signals usually start with `u` and control output signals usually start with `y`, unless use of the physical quantity is clearer.
6. The following strings are frequently used:
   - Prefix `use_` for conditionally enabled input signals, such as `use_T_in` for enabling an input connector for temperature in Buildings.Fluid.Sources.Boundary_pT, or as `use_enthalpy` in Buildings.Controls.OBC.ASHRAE.G36.AHUs.MultiZone.VAV.Economizers.Subsequences.Enable to flag if it should evaluate outdoor air enthalpy in addition to temperature.
   - Prefix `have_` if a controller has a certain input, such as `have_CO2Sen` in Buildings.Controls.OBC.ASHRAE.G36.TerminalUnits.Reheat.Controller if the zone has a CO2 sensor.
   - Suffix `_flow` for a flow variable, such as `Q_flow`, `m_flow` and `V_flow`. See Buildings.Fluid.Sensors.VolumeFlowRate.
   - Suffix `_nominal` for the design or nominal capacity, i.e., `Q_flow_nominal` is the capacity of a device that it has at full load, and `m_flow_nominal` is the design mass flow rate. See Buildings.Fluid.HeatExchangers.HeaterCooler_u.
   - Suffix `_small` for a small value which is typically used for regularization (to ensure a numerically robust implementation).
   - Suffix `Set` for set point.
   - Suffix `_min` (`_max`) for minimum (maximum), such as `TSupCoo_min` for the lowest cooling supply air temperature setpoint in Buildings.Controls.OBC.ASHRAE.G36.AHUs.MultiZone.VAV.SetPoints.SupplyTemperature.
7. The two connectors of a domain that have identical declarations and different icons are usually distinguished by `_a`, `_b` or `_p`, `_n`. Examples are fluid ports `port_a` and `port_b` or electrical connectors `terminal_p` and `terminal_n`.

**claude-opus-4-6-v1**

Thought for 1 seconds

Got it — I've captured the first set of rules. These appear to be naming conventions for a Modelica-based library (likely the Modelica Buildings Library or similar).

Go ahead and share the next set(s) of rules, and once I have them all, I'll:

1. Merge everything into a single unified set
2. Call out any contradictions, ambiguities, or overlaps

Ready when you are!

The second set of rules are as follows:

## Information

The `Buildings.Controls.OBC` package follows the naming conventions of the `Buildings` Library, see Buildings.UsersGuide.Conventions. The table below shows some examples of commonly used names. Note that the names are generally composed as follows:

- If needed to understand the context, a prefix `u` for input or `y` for output may be used.
- Next, the quantity is used, such as `TOut` or `TZonHea`.
- Finally, as a postfix a qualifier may be added, such as `Set` for set point, `Min` for minimum, or `Coo` for cooling.

Generally, we strive for short names, and therefore often prefix or postfix are omitted if the type of the variable is clear from the context. For example, a room temperature thermostat may simply use `T` as an input as it is clear that this will be the room temperature.

| INSTANCE NAMES |
| --- |
| **Name** |
| `TOut` ( `hOut` ) |
| `TZonHeaSet` ( `TZonCooSet` ) |
| `VDis_flow` |
| `dpBui` |
| `uOpeMod` |
| `uResReq` |
| `uSupFan` |
| `uSupFanSpe` |
| `uDam` |
| `uHea` ( `uCoo` ) |
| `yPosMin` ( `yPosMax` ) |

| |
|---|
| `yHeaCoi` ( `yCooCoi` ) |
| `uChiIsoVal` |
| `dpChiWat_remote` |
| `TChiWatRet` |
| `VChiWat_flow` |
| `uHeaPreCon` |
| `yChiPumSpe` |
| `yChi` |
| `yMinValPos` |

**Parameter names**

| **Name** |
|---|
| `use_TMix` |
| `have_occSen` ( `have_winSen` ) |
| `AFlo` |
| `VDisHeaSetMax_flow` ( `VDisCooSetMax_flow` ) |
| `VOutPerAre_flow` ( `VOutPerPer_flow` ) |
| `V_flow_nominal` |
| `VOutMin_flow` |
| `pMinSet` ( `pMaxSet` ) |
| `TSupSetMin` ( `TSupSetMax` ) |
| `TOccHeaSet` ( `TUnoHeaSet` ) |
| `TZonCooMax` ( `TZonCooMin` ) |
| `retDamPhyPosMax` ( `outDamPhyPosMax` ) |
| `samplePeriod` |
| `zonDisEffHea` ( `zonDisEffCoo` ) |
| `kCoo` |
| `TiCoo` |
| `TdCoo` |
| `nChi` |
| `have_parChi` |
| `have_heaChiWatPum` |
| `have_fixSpeConWatPum` |
| `need_reduceChillerDemand` |
| `minChiLif` |

`TChiWatSupMin`

`dpChiWatPumMin`

`lift_min`

**claude-opus-4-6-v1**

Thought for 1 seconds

Got it — second set captured. This one adds more detailed naming conventions specifically for the `Buildings.Controls.OBC` package, with extensive examples for instance names and parameter names.

I'm already noticing some areas of overlap and a few things I'll want to reconcile (e.g., suffix conventions like `Set` vs `_min` / `_max` capitalization patterns, the `use_` / `have_` prefix conventions appearing in both sets, etc.).

Do you have more sets to share, or should I go ahead and consolidate these two into a unified set now?

The third set of rules are here

**STANDARD CHECKING PROCESSES BEFORE PULL REQUEST:**

1. Add comments to all the instances: parameters, blocks, instances; If description of function is obvious, just use names like “Logical And”, etc. Make sure comments are short and don’t contain any redundant information.

2. Text sections are in right order:

1. ->parameter

2. -> interface (inputs, outputs, ports)

3. -> blocks/instances

4. -> (protected sections, make sure to protect correct blocks-they won’t be accessible in the plots);

3. Ensure readability: make sure parameter assignment per line;

4. Declare inputs/outputs/parameters:

final unit, quantity;

Any assignments that should not be editable by user should have keyword final.

5. Avoid blank lines: use gedit or atom to delete those blank lines (leave one blank line between instances if you like for readability);

6. Ensure right naming, block names and variables usually start with a lowercase letter, unless for example a temperature. If the name has modifiers like setpoint, lockout, minimum, maximum, etc. it should be appended after the variable name.

7. Should have “%name” at icon layer

8. Should have default component name for each block;

9. Should have information section in package.mo file;

10. Ensure correct revision record;

11. Use reference section;

12. Need to have validation package;

13. Done unit tests;

14. Ensure there is no typo;

15. Search for “public” to check no elements have been added after protected

16. Check the “Files changed” before asking for review: make sure there is no unnecessary change to original files;

17. Read through the Info section, check images for typos

18. Make sure any referenced blocks and models are referenced using a hyperlink and not the model name.

19. Use full paths for all blocks: e.g. Buildings.Controls...

20. Pull fresh master branch or staging branch and merge it in

21. Location of the module in the whole library

22. Check for consolidation with existing module

23. Submitted to the LBNL/Buildings library on GitHub

24. Try to avoid descriptors in parameter and instance names that may be obvious already from the package. For example, avoid boi for boiler and pla for plant if the blocks are in the BoilerPlant sequence package.

25. If there is an analogous sequence for other systems, make sure you duplicate names for ease of commonization. Also ensure parameters are arranged in identical/similar tabs.

26. Change instance names on feedback delay pre blocks from “pre” to “pre1”, etc.

27. In the validation file, ensure instances being tested are at the top of the model.

28. Add a blank line at the end of each .mos script

29. If a parameter is used to define another parameter in the same block, ensure the two parameters appear one after another

30. All instances must be sorted by data type (boolean > integer > real) within their instance type (parameter > input > output > protected constants > protected blocks > equations > annotation). Try to follow the same order at the graphical and icon layer.

31. Ensure all related instances (for eg. measured quantity and related hysteresis deadband) have the same units for consistency.

32. Ensure all variable names appear right after the text related to it.

33. Ensure check is performed for ALL files being committed (including validation files and package files being committed for updating package hierarchy).

34. Run verifyFiles.py to check if there is any format issue.

35. Check and remove any redundant zeros in the .mos script.

36. Check if there is any BOM being introduced and remove any of them

**claude-opus-4-6-v1**

Thought for less than a second

Got it — third set captured. This one is more of a **pre-pull-request checklist** covering code quality, ordering, formatting, validation, and review processes, in addition to some naming rules.

I can already see several areas to reconcile across all three sets — for example:

- **Rule 6 in Set 3** says "block names and variables usually start with a lowercase letter, unless for example a temperature" — which may partially conflict with **Rule 1 in Set 1** ("Class names of models, blocks and packages should start with an upper-case letter")
- Overlap in naming suffix conventions ( `Set` , `_min` / `_max` , `use_` / `have_` , `_flow` , etc.)
- The `_nominal` vs `_min` / `_max` suffix casing conventions

Do you have any more sets to share, or should I go ahead and consolidate these three into a single unified set with contradiction/ambiguity notes?

While consolidating the rules, I want you to follow certain instructions.

1. Filter out only the naming related rules from set 3.
2. Set 1 and set 2 are the two primary sources. Set 3 is a secondary source. If any naming rules in set 3 contradict rules in the primary sources, ignore the rule from set 3.

Let me know if you have any questions.

**claude-opus-4-6-v1**

Thought for 2 minutes

# Consolidated Naming Convention Rules

Below is the unified rule set, followed by a dedicated section on contradictions and ambiguities.

---

## 1. Class Names (Models, Blocks, Packages)

**1.1** Class names shall start with an **uppercase letter** and be a noun, or a combination of adjectives and nouns.

**1.2** Use **camelCase** to combine multiple words (e.g., `HeatTransfer`).

**1.3** Do not repeat higher-level package names. Use `Chillers.Carnot`, not `Chillers.CarnotChiller`. Similarly, avoid descriptors in names that are already obvious from the enclosing package (e.g., avoid `boi` for boiler if the block is already in a `BoilerPlant` package).

> *"Sources: Set 1 Rule 1, Set 3 Rule 24"*

---

## 2. Instance and Variable Names

### 2.1 General Structure

**2.1** Instance names (variables, block instances, parameters) shall generally start with a **lowercase letter**, unless they represent a physical quantity conventionally capitalized (e.g., `T` for temperature, `P` for power, `X` for mass fraction).

**2.2** Names are composed following this pattern:

| POSITION | ELEMENT | EXAMPLES |
|---|---|---|
| 1. Prefix *(optional)* | `u` for input, `y` for output — include only if needed for context | `uOpeMod`, `yHeaCoi` |
| 2. Quantity *(required)* | The physical quantity or descriptor | `TOut`, `TZonHea`, `dpBui` |
| 3. Postfix/Qualifier *(optional)* | Modifiers such as setpoint, min, max, cooling, heating, etc., appended **after** the quantity | `TZonHeaSet`, `TSupSetMin` |

**2.3** Strive for **short names**. Omit prefix or postfix when the meaning is clear from context. For example, a room thermostat may simply use `T` as its input.

> *"Sources: Set 1 Rules 2 & 5, Set 2 general conventions, Set 3 Rule 6"*

### 2.2 Abbreviation Conventions

**2.4** Use abbreviations composed of approximately the **first three characters** of each word, combined in camelCase (e.g., `preDro` for pressure drop, `worTim` for working timer, `zonDisEffHea` for zone distribution effectiveness heating).

**2.5** Single characters may be used where generally understood (e.g., `T`, `p`, `k`, `n`, `u`, `y`). These may be augmented with descriptors as needed (e.g., `yVal` for valve signal, `yDam` for damper

signal, `kCoo` for cooling gain).

> *"Sources: Set 1 Rule 2, Set 2 examples"*

### 2.3 Cross-Package Consistency

**2.6** If an analogous sequence exists for another system (e.g., heating vs. cooling, chiller plant vs. boiler plant), use **consistent, parallel names** across both to facilitate commonization.

**2.7** For feedback delay `pre` block instances, use **numbered names** (`pre1`, `pre2`, etc.) rather than bare `pre`.

> *"Sources: Set 3 Rules 25 & 26"*

---

## 3. Standard Physical Quantity Variables

**3.1** The following single-character or short variables are standard:

| VARIABLE | MEANING |
|---|---|
| `T` | Temperature |
| `p` | Pressure |
| `dp` | Pressure difference |
| `P` | Power |
| `E` (or `Q`) | Energy (or thermal energy) |
| `X` | Mass fraction per total mass |
| `x` | Mass fraction per mass of dry air |
| `Q_flow` | Heat flow rate |
| `m_flow` | Mass flow rate |
| `H_flow` | Enthalpy flow rate |
| `V_flow` | Volume flow rate |
| `k` | Gain (e.g., `kCoo`) |
| `Ti` | Integrator time constant (e.g., `TiCoo`) |
| `Td` | Derivative time constant (e.g., `TdCoo`) |
| `n` | Count / number (e.g., `nChi` for number of chillers) |
| `A` | Area (e.g., `AFlo` for floor area) |
| `h` | Specific enthalpy (e.g., `hOut` for outdoor air enthalpy) |

> *"Sources: Set 1 Rules 3–4, Set 2 tables"*

---

## 4. Control Signal Naming

**4.1** Control **input** signals start with `u`; control **output** signals start with `y`.

**4.2** The `u` / `y` prefix may be **omitted** when the physical quantity already makes the direction clear.

**4.3** When a controller has multiple outputs or inputs, augment with a descriptor (e.g., `yVal` and `yDam` for valve and damper output signals; `uHea` and `uCoo` for heating and cooling input signals).

> *"Sources: Set 1 Rules 2 & 5, Set 2 conventions"*

---

# 5. Prefix Conventions

**5.1** The following prefixes are standard:

| PREFIX | MEANING | EXAMPLES |
|---|---|---|
| `u` | Control input signal | `uOpeMod`, `uDam`, `uHea`, `uCoo` |
| `y` | Control output signal | `yHeaCoi`, `yCooCoi`, `yPosMin`, `yChiPumSpe` |
| `use_` | Conditionally enabled input or feature (boolean parameter) | `use_T_in`, `use_TMix`, `use_enthalpy` |
| `have_` | Indicates the system/zone has a particular component, sensor, or configuration (boolean parameter) | `have_CO2Sen`, `have_occSen`, `have_winSen`, `have_parChi` |
| `need_` | Indicates a required capability or constraint (boolean parameter) | `need_reduceChillerDemand` |

> *"Sources: Set 1 Rule 6, Set 2 tables"*

---

# 6. Suffix / Postfix Conventions

**6.1** The following suffixes are standard:

| SUFFIX | MEANING | EXAMPLES |
|---|---|---|
| `_flow` | Flow variable | `Q_flow`, `m_flow`, `V_flow`, `VDis_flow` |
| `_nominal` | Design / nominal capacity | `Q_flow_nominal`, `m_flow_nominal`, `V_flow_nominal` |
| `_small` | Small value used for regularization | — |
| `Set` | Setpoint | `TZonHeaSet`, `TZonCooSet` |
| `Min` / `Max` | Minimum / Maximum | `TSupSetMin`, `TZonCooMax`, `VDisHeaSetMax_flow` |
| `_remote` | From a remote sensor | `dpChiWat_remote` |
| `Hea` / `Coo` | Heating / Cooling qualifier | `TZonHeaSet`, `yCooCoi`, `zonDisEffCoo` |

**6.2** When multiple suffixes are needed, follow this general order: *Quantity → Context/Qualifier → Setpoint → Min/Max →* `_flow`. For example: `VDisHeaSetMax_flow`.

> *"Sources: Set 1 Rule 6, Set 2 tables and examples"*

---

# 7. Connector Naming

**7.1** Paired connectors of the same domain (identical declarations, different icons) are distinguished by:

| CONVENTION | USAGE | EXAMPLE |
|---|---|---|
| `_a` / `_b` | Fluid ports | `port_a`, `port_b` |

| | | |
|---|---|---|
| `_p` / `_n` | Electrical terminals | `terminal_p` , `terminal_n` |

> *"Source: Set 1 Rule 7"*

---

## 8. Psychrometrics Package Naming (`Buildings.Utilities.Psychrometrics`)

**8.1** Uppercase `X` = mass fraction per total mass; lowercase `x` = mass fraction per mass of dry air.

**8.2** The notation `z_xy` denotes a function or block with output `z` and inputs `x` and `y`.

**8.3** Standard symbols:

| SYMBOL | MEANING |
|---|---|
| `pW` | Water vapor pressure |
| `TDewPoi` | Dew point temperature |
| `TWetBul` | Wet bulb temperature |
| `TDryBul` (or `T`) | Dry bulb temperature |

> *"Source: Set 1 Rule 4"*

---

# Contradictions and Ambiguities

### Contradiction 1: `_min` / `_max` vs. `Min` / `Max`

| SET 1 (EXPLICIT RULE) | SET 2 (EXAMPLES) |
|---|---|
| Suffix `_min` / `_max` (underscore + lowercase), e.g., `TSupCoo_min` | Suffix `Min` / `Max` (camelCase, no underscore), e.g., `TSupSetMin` , `TZonCooMax` , `dpChiWatPumMin` |

Set 2 states that it follows the conventions of Set 1, yet its examples overwhelmingly use `Min` / `Max` in camelCase without an underscore. Confusingly, Set 2 also contains `lift_min` , which follows the Set 1 convention.

**Recommendation:** Adopt **one** format and apply it consistently. The `Min` / `Max` camelCase style is more consistent with other postfix qualifiers ( `Set` , `Hea` , `Coo` ) that also have no underscore. If that is chosen, the Set 1 rule and the `lift_min` example in Set 2 should be updated.

---

### Contradiction 2: `min` used as a prefix vs. postfix

Set 2 contains the parameter `minChiLif` (minimum chiller lift), where `min` appears as a **prefix**. This directly contradicts:

- Set 1 Rule 6 (suffixes `_min` / `_max` )
- Set 2's own stated convention ("as a postfix a qualifier may be added, such as... `Min` for minimum")
- Set 3 Rule 6 ("modifiers like... minimum, maximum... should be appended **after** the variable name")

**Recommendation:** Rename `minChiLif` to something like `chiLifMin` or `lift_min` to follow the postfix convention.

---

## Ambiguity 1: `need_` prefix not formally defined

The prefix `need_` appears in Set 2's examples (`need_reduceChillerDemand`) but is never explicitly defined as a convention in either Set 1 or Set 2's descriptive text (unlike `use_` and `have_`, which are). Its intended semantic distinction from `have_` is unclear.

**Recommendation:** Formally define `need_` and clarify how it differs from `have_`. A possible distinction: `have_` = "the physical system includes this" vs. `need_` = "the control logic requires this behavior."

---

## Ambiguity 2: When to use underscores vs. camelCase in suffixes

Some suffixes use an underscore (`_flow`, `_nominal`, `_small`, `_remote`) while others use camelCase with no underscore (`Set`, `Min`, `Max`, `Hea`, `Coo`). No explicit rule governs which style to use.

**Possible implicit pattern:** Underscored suffixes (`_flow`, `_nominal`, `_small`) tend to be **category markers** that describe the *type* of quantity, while camelCase suffixes (`Set`, `Min`, `Hea`) tend to be **qualifiers** of the specific value. However, `_remote` doesn't fit this pattern neatly.

**Recommendation:** Explicitly state the rule. For example: *"Use underscore suffixes for quantity-type markers (`_flow`, `_nominal`, `_small`). Use camelCase postfixes for value qualifiers (`Set`, `Min`, `Max`, `Hea`, `Coo`)."*

---

## Ambiguity 3: When to abbreviate vs. use full words

Set 1 Rule 2 says to use "the first three characters of a word," and most examples follow this (`Hea`, `Coo`, `Dis`, `Sup`). However, Set 2 contains full-word names like `samplePeriod`, `lift_min`, and `retDamPhyPosMax`. No rule clarifies when full words are acceptable or preferred.

**Recommendation:** Add guidance such as: *"Use three-character abbreviations for domain-specific or frequently used terms. Full words may be used for general programming terms or when abbreviation would be ambiguous."*

---

## Ambiguity 4: Ordering of multiple postfix qualifiers

When a name requires multiple qualifiers, the ordering is inconsistent across Set 2's examples:

| NAME | APPARENT ORDER |
|---|---|
| `TSupSetMin` | Quantity → Set → Min |
| `pMinSet` | Quantity → Min → Set |
| `VDisHeaSetMax_flow` | Quantity → Context → Set → Max → _flow |

`TSupSetMin` puts `Set` before `Min`, while `pMinSet` puts `Min` before `Set`.

**Recommendation:** Standardize the order explicitly. A suggested canonical order: **Quantity → Context (`Hea` / `Coo` / `Dis` /...) → `Set` → `Min` / `Max` → `_flow` / `_nominal`.**

---

## Ambiguity 5: Set 3 Rule 6 — "block names" interpretation

Set 3 Rule 6 states *"block names and variables usually start with a lowercase letter."* If "block names" refers to **class names**, this contradicts Set 1 Rule 1 (class names start uppercase). If it refers to **instance names** of blocks, it is consistent. Per the user's instructions, if it is a contradiction with Set 1, it is ignored. The consolidated rules above interpret it as referring to instance names.

---

Let me know if you'd like me to adjust the organization, add more examples, or resolve any of the flagged contradictions with a specific decision.

Can you format the output as a md file?

**claude-opus-4-6-v1**

Here's the content as a downloadable Markdown file:

markdown ◇ Collapse Copy

```
# Consolidated Naming Convention Rules

Below is the unified rule set, followed by a dedicated section on contradictions and ambiguities

---

## 1. Class Names (Models, Blocks, Packages)

**1.1** Class names shall start with an **uppercase letter** and be a noun, or a combination of

**1.2** Use **camelCase** to combine multiple words (e.g., `HeatTransfer`).

**1.3** Do not repeat higher-level package names. Use `Chillers.Carnot`, not `Chillers.CarnotCh

> *Sources: Set 1 Rule 1, Set 3 Rule 24*

---
```

## 2. Instance and Variable Names

### 2.1 General Structure

**2.1.1** Instance names (variables, block instances, parameters) shall generally start with a

**2.1.2** Names are composed following this pattern:

| Position | Element | Examples |
|---|---|---|
| 1. Prefix *(optional)* | `u` for input, `y` for output — include only if needed for context |
| 2. Quantity *(required)* | The physical quantity or descriptor | `TOut`, `TZonHea`, `dpBui` |
| 3. Postfix/Qualifier *(optional)* | Modifiers such as setpoint, min, max, cooling, heating, et

**2.1.3** Strive for **short names**. Omit prefix or postfix when the meaning is clear from cont

> *Sources: Set 1 Rules 2 & 5, Set 2 general conventions, Set 3 Rule 6*

### 2.2 Abbreviation Conventions

**2.2.1** Use abbreviations composed of approximately the **first three characters** of each wor

**2.2.2** Single characters may be used where generally understood (e.g., `T`, `p`, `k`, `n`, `u

> *Sources: Set 1 Rule 2, Set 2 examples*

### 2.3 Cross-Package Consistency

**2.3.1** If an analogous sequence exists for another system (e.g., heating vs. cooling, chiller

**2.3.2** For feedback delay `pre` block instances, use **numbered names** (`pre1`, `pre2`, etc

> *Sources: Set 3 Rules 25 & 26*

---

## 3. Standard Physical Quantity Variables

**3.1** The following single-character or short variables are standard:

| Variable | Meaning |

| Variable | Meaning |
|---|---|
| `T` | Temperature |
| `p` | Pressure |
| `dp` | Pressure difference |
| `P` | Power |
| `E` (or `Q`) | Energy (or thermal energy) |
| `X` | Mass fraction per total mass |
| `x` | Mass fraction per mass of dry air |
| `Q_flow` | Heat flow rate |
| `m_flow` | Mass flow rate |
| `H_flow` | Enthalpy flow rate |
| `V_flow` | Volume flow rate |
| `k` | Gain (e.g., `kCoo`) |
| `Ti` | Integrator time constant (e.g., `TiCoo`) |
| `Td` | Derivative time constant (e.g., `TdCoo`) |
| `n` | Count / number (e.g., `nChi` for number of chillers) |
| `A` | Area (e.g., `AFlo` for floor area) |
| `h` | Specific enthalpy (e.g., `hOut` for outdoor air enthalpy) |

> *Sources: Set 1 Rules 3–4, Set 2 tables*

---

## 4. Control Signal Naming

**4.1** Control **input** signals start with `u`; control **output** signals start with `y`.

**4.2** The `u`/`y` prefix may be **omitted** when the physical quantity already makes the direc

**4.3** When a controller has multiple outputs or inputs, augment with a descriptor (e.g., `yVa

> *Sources: Set 1 Rules 2 & 5, Set 2 conventions*

---

## 5. Prefix Conventions

**5.1** The following prefixes are standard:

| Prefix | Meaning | Examples |

|---|---|---|

| `u` | Control input signal | `uOpeMod`, `uDam`, `uHea`, `uCoo` |

| `y` | Control output signal | `yHeaCoi`, `yCooCoi`, `yPosMin`, `yChiPumSpe` |

| `use_` | Conditionally enabled input or feature (boolean parameter) | `use_T_in`, `use_TMix`,

| `have_` | Indicates the system/zone has a particular component, sensor, or configuration (boo

| `need_` | Indicates a required capability or constraint (boolean parameter) | `need_reduceChi

> *Sources: Set 1 Rule 6, Set 2 tables*

---

## 6. Suffix / Postfix Conventions

**6.1** The following suffixes are standard:

| Suffix | Meaning | Examples |

|---|---|---|

| `_flow` | Flow variable | `Q_flow`, `m_flow`, `V_flow`, `VDis_flow` |

| `_nominal` | Design / nominal capacity | `Q_flow_nominal`, `m_flow_nominal`, `V_flow_nominal`

| `_small` | Small value used for regularization | – |

| `Set` | Setpoint | `TZonHeaSet`, `TZonCooSet` |

| `Min` / `Max` | Minimum / Maximum | `TSupSetMin`, `TZonCooMax`, `VDisHeaSetMax_flow` |

| `_remote` | From a remote sensor | `dpChiWat_remote` |

| `Hea` / `Coo` | Heating / Cooling qualifier | `TZonHeaSet`, `yCooCoi`, `zonDisEffCoo` |

**6.2** When multiple suffixes are needed, follow this general order: *Quantity → Context/Quali

> *Sources: Set 1 Rule 6, Set 2 tables and examples*

---

## 7. Connector Naming

**7.1** Paired connectors of the same domain (identical declarations, different icons) are dist

| Convention | Usage | Example |

|---|---|---|

| `_a` / `_b` | Fluid ports | `port_a`, `port_b` |

| `_p` / `_n` | Electrical terminals | `terminal_p`, `terminal_n` |

> *Source: Set 1 Rule 7*

---

## 8. Psychrometrics Package Naming (`Buildings.Utilities.Psychrometrics`)

**8.1** Uppercase `X` = mass fraction per total mass; lowercase `x` = mass fraction per mass of

**8.2** The notation `z_xy` denotes a function or block with output `z` and inputs `x` and `y`.

**8.3** Standard symbols:

| Symbol | Meaning |
|---|---|
| `pW` | Water vapor pressure |
| `TDewPoi` | Dew point temperature |
| `TWetBul` | Wet bulb temperature |
| `TDryBul` (or `T`) | Dry bulb temperature |

> *Source: Set 1 Rule 4*

---

---

# Contradictions and Ambiguities

### Contradiction 1: `_min`/`_max` vs. `Min`/`Max`

| Set 1 (explicit rule) | Set 2 (examples) |
|---|---|
| Suffix `_min` / `_max` (underscore + lowercase), e.g., `TSupCoo_min` | Suffix `Min` / `Max` (

Set 2 states that it follows the conventions of Set 1, yet its examples overwhelmingly use `Min`

**Recommendation:** Adopt **one** format and apply it consistently. The `Min`/`Max` camelCase st

---

### Contradiction 2: `min` used as a prefix vs. postfix

Set 2 contains the parameter `minChiLif` (minimum chiller lift), where `min` appears as a **pre

- Set 1 Rule 6 (suffixes `_min`/`_max`)
- Set 2's own stated convention ("as a postfix a qualifier may be added, such as… `Min` for min
- Set 3 Rule 6 ("modifiers like… minimum, maximum… should be appended **after** the variable na

**Recommendation:** Rename `minChiLif` to something like `chiLifMin` or `lift_min` to follow the

---

### Ambiguity 1: `need_` *prefix not formally defined*

*The prefix `need_` appears in Set 2's examples (`need_reduceChillerDemand`) but is never explic*

**Recommendation:** Formally define `need_` and clarify how it differs from `have_`. A possible

---

### Ambiguity 2: When to use underscores vs. camelCase in suffixes

Some suffixes use an underscore (`_flow`, `_nominal`, `_small`, `_remote`) while others use came

**Possible implicit pattern:** Underscored suffixes (`_flow`, `_nominal`, `_small`) tend to be

**Recommendation:** Explicitly state the rule. For example: *"*Use underscore suffixes for quant

---

### Ambiguity 3: When to abbreviate vs. use full words

Set 1 Rule 2 says to use "the first three characters of a word," and most examples follow this

**Recommendation:** Add guidance such as: *"*Use three-character abbreviations for domain-specif

---

### Ambiguity 4: Ordering of multiple postfix qualifiers

When a name requires multiple qualifiers, the ordering is inconsistent across Set 2's examples:

```
| Name | Apparent order |
|---|---|
| `TSupSetMin` | Quantity → Set → Min |
| `pMinSet` | Quantity → Min → Set |
| `VDisHeaSetMax_flow` | Quantity → Context → Set → Max → \_flow |

`TSupSetMin` puts `Set` before `Min`, while `pMinSet` puts `Min` before `Set`.

**Recommendation:** Standardize the order explicitly. A suggested canonical order: **Quantity →

---

### Ambiguity 5: Set 3 Rule 6 — "block names" interpretation

Set 3 Rule 6 states *"block names and variables usually start with a lowercase letter."* If "bl
```

You can copy the above raw Markdown into any `.md` file. Let me know if you'd like any adjustments!

Thanks!

**claude-opus-4-6-v1**

You're welcome! If you ever need to revisit or refine these rules — or if you get additional rule sets to integrate — feel free to reach out. Good luck with the project! 😊